
OnDemand Blender MCP - Setup & Architecture

Wiring ahujasad/blender-mcp v1.5.6 into an OnDemand Agent via an HTTPS bridge |
REGISTRATION STATUS: FAILED - POST /plugin/v1 returned HTTP 400
invalid_request, no pluginId issued

Contents

1. Executive Summary	3
2. Architecture	4
2.1 Amended Architecture Diagram	5
3. The Five-Hop Call Chain	6
3.1 Security and Call-Sequence Diagram	7
4. The 14 Tools	9
5. Three-Step Mac Setup	10
6. The HTTPS Bridge	11
7. OnDemand Registration	12
8. Fact-Check Table	13
10. Live Registration Record	14
11. Security Hardening	16
12. Troubleshooting	18
13. Sources	20

1. Executive Summary

OnDemand's public documentation contains **no native MCP (Model Context Protocol) server registration capability**. This was verified exhaustively rather than assumed: a keyword scan for mcp across all 28 fetched OpenAPI documents returned **0 occurrences**, as did a scan of the official machine-readable docs index llms.txt. Speculative doc slugs (mcp, createtool, plugins, suggestplugins) all returned HTTP 400 invalid request - they do not exist.

There is no endpoint that accepts an MCP server.json manifest, an MCP server URL, a stdio command such as uvx blender-mcp, or an MCP tools/list capability document. There is no transport negotiation, no Mcp-Session-Id handling and no tools/call proxy.

The documented alternative - and it is a good one

OnDemand's equivalent of "register a tool" is the **REST API Agent** (a plugin in the API layer). You describe your tool surface as an **OpenAPI 3.0 schema**, register it as a plugin, receive a **pluginId** of the form plugin-, and pass that ID in the pluginIds array when creating a chat session or submitting a query. You are **re-expressing** the MCP tool surface as REST operations - you are not connecting an MCP server, because the platform has no documented way to consume one.

Fact	Value
Upstream project	ahujasid/blender-mcp
Licence	MIT - actively maintained
Stars (verified)	~22.7k
Latest release	PyPI blender-mcp v1.5.6, 18 Mar 2026
addon.py last touched	23 Jan 2026
Prerequisites	Blender 3.0+, Python 3.10+, uv package manager
Integration shape	MCP tool surface re-expressed as an OpenAPI 3.0 REST plugin
Registration outcome	FAILED - HTTP 400, no pluginId (see S10)

2. Architecture

blender-mcp runs as **two local processes**. The **Blender addon** (addon.py) opens a **TCP socket server on localhost:9876** inside the live Blender application. The **MCP server** (src/blender_mcp/server.py, launched with uvx blender-mcp) speaks **MCP over stdio** to a local client such as Claude Desktop or Cursor. Neither transport is reachable by a hosted service.

blender-mcp transport	Callable by OnDemand?	Why / why not
stdio (uvx blender-mcp)	No	OnDemand is hosted; it cannot spawn a process on your machine
Raw TCP localhost:9876	No	Not HTTP, not TLS, not routable, bound to loopback
HTTPS REST (your bridge)	Yes	The only shape a REST API Agent's OpenAPI schema can describe

A small HTTPS service - **the bridge** - is therefore mandatory. It accepts REST calls and forwards them as blender-mcp JSON commands over the TCP socket. For development, a tunnel (ngrok / Cloudflare Tunnel) in front of a local wrapper; for production, a hosted wrapper alongside a headless Blender (blender --background).

Non-negotiable bridge requirements

- **Public HTTPS with a valid certificate.** The servers[].url in your schema must be an absolute https:// URL OnDemand's servers can resolve. localhost, 127.0.0.1 and RFC-1918 addresses will never work.
- **Stable hostname.** A free-tier tunnel that rotates its subdomain silently breaks the plugin - you would have to re-register the schema each time.
- **Respond within the tool timeout.** plugin_execution_timeout (HTTP 504) fires at **2.5 minutes**. Long renders must be asynchronous: return a job ID and poll.

2.1 Amended Architecture Diagram

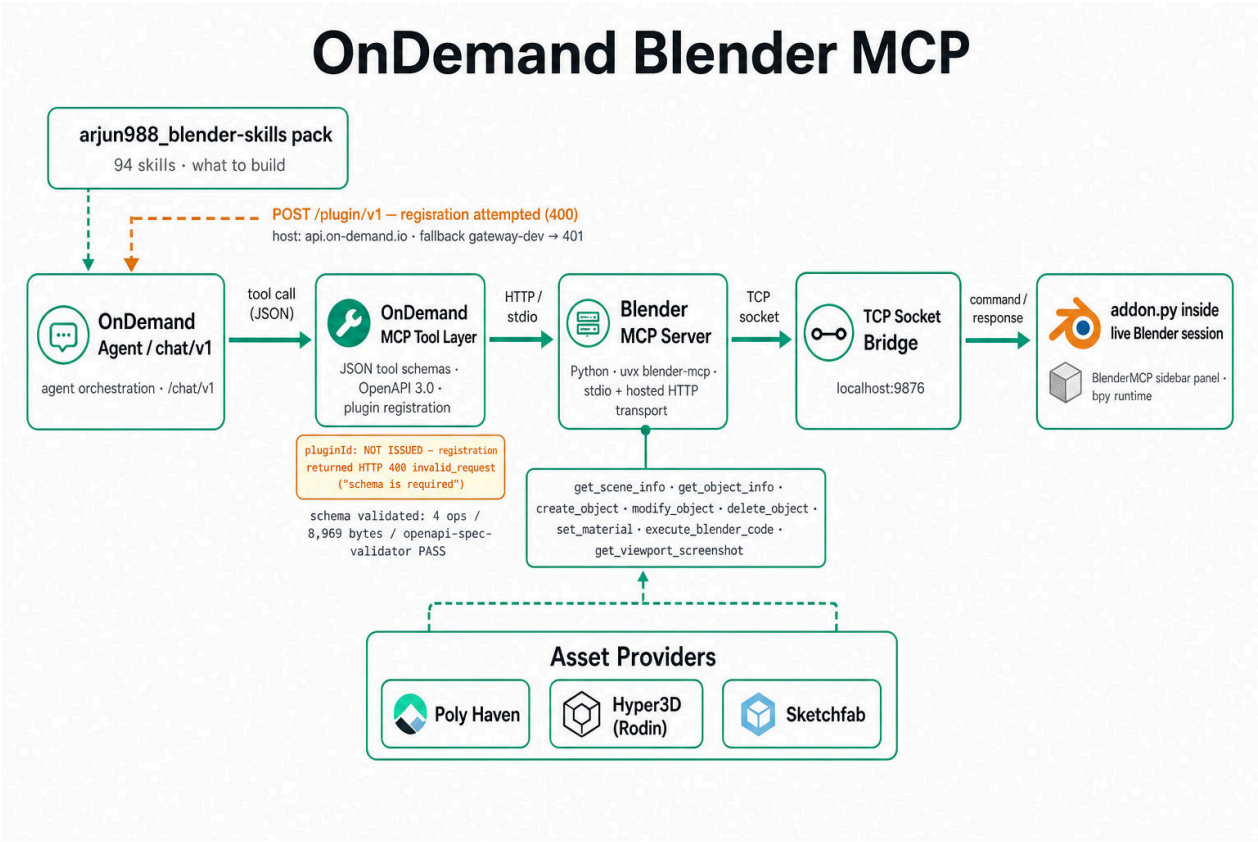


Figure 1 - OnDemand ⇄ Blender MCP bridge architecture (amended)
Supersedes the earlier architecture figure. Source: this guide, S6-S7.

3. The Five-Hop Call Chain

Every tool invocation traverses exactly five hops. Understanding the boundary at each hop is what makes the security model in **S11** legible.

Hop	From	To	Transport	What happens
1	OnDemand Agent	OnDemand chat session / query	HTTPS (OnDemand internal)	Model decides a tool is needed; pluginIds must be present or the RAG layer is bypassed
2	OnDemand tool / plugin layer	Your bridge	HTTPS + apikey / X-Bridge-Key header	OpenAPI operation is resolved to a concrete REST call; 2.5 min timeout applies
3	MCP server / bridge wrapper	Local translation layer	In-process	REST payload is translated into a blender-mcp JSON command
4	TCP socket :9876	localhost only	Raw TCP, JSON command protocol	Loopback-bound; never bind 0.0.0.0
5	addon.py in live Blender	bpy runtime	In-process Python	Command is executed against the live scene; result is returned back up the chain

```
OnDemand Agent
  | 1. pluginIds -> tool selection
OnDemand tool / plugin layer
  | 2. HTTPS + auth header (public internet)
MCP server / HTTPS bridge
  | 3. REST -> blender-mcp JSON command
TCP socket 127.0.0.1:9876
  | 4. loopback only
addon.py inside live Blender (bpy)
```

Hop 2 is the only hop that crosses the public internet, and it is therefore the only hop where TLS, origin validation and the shared secret can be enforced. Hops 3-5 are entirely on your host - which is precisely why the Blender socket must never leave loopback.

3.1 Security and Call-Sequence Diagram

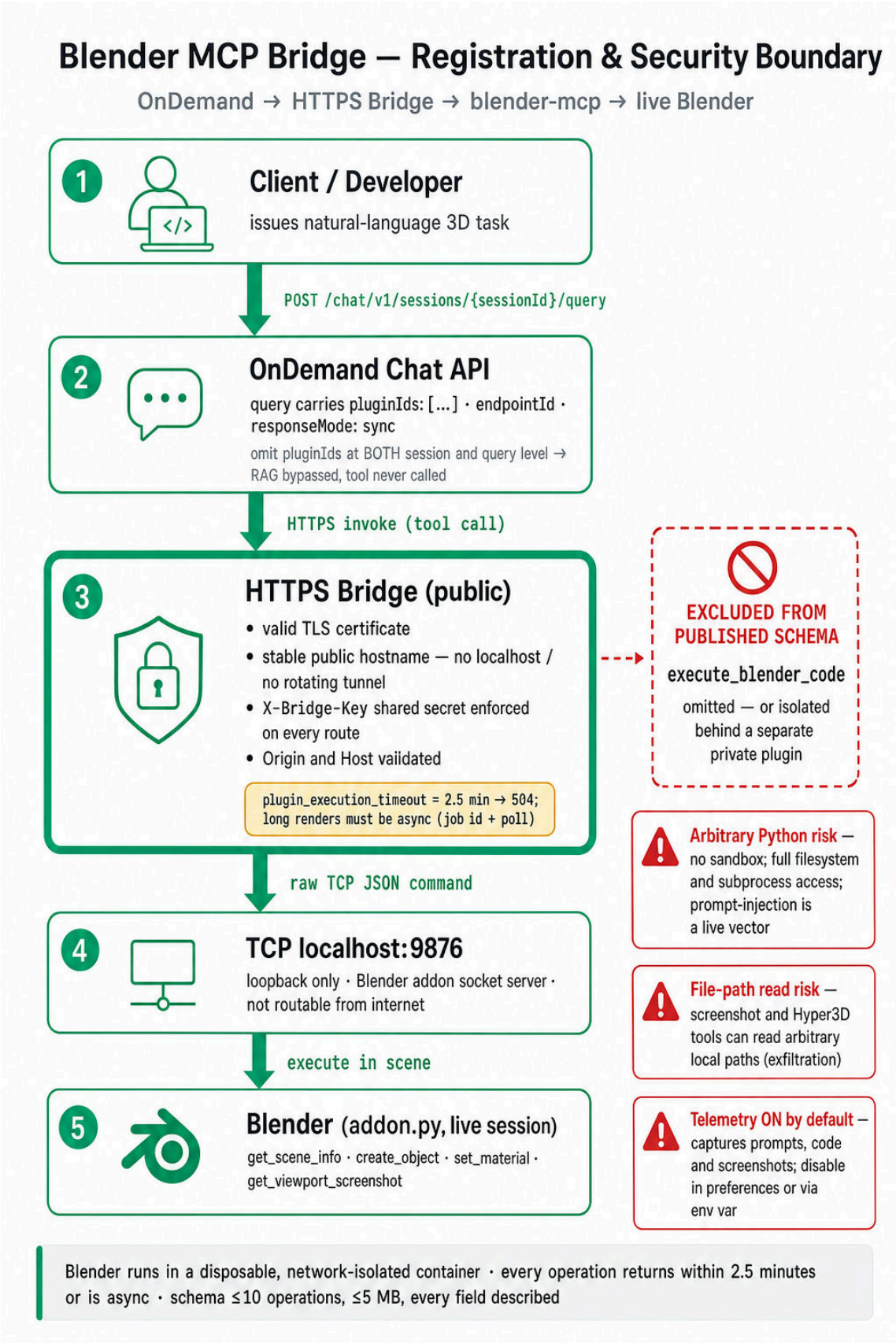


Figure 2 - Security and call-sequence view: agent → tool layer → MCP server → TCP :9876 → Blender
Trust boundaries annotated per S11. Source: this guide.

4. The 14 Tools

The upstream blender-mcp server exposes fourteen tools. Four are mapped into the reference OpenAPI schema in [S7](#); the rest can be added the same way, staying within OnDemand's **10-operation publishing limit**.

#	Tool	Capability	REST mapping	In reference schema
1	get_scene_info	Scene inspection	GET /scene	Yes
2	get_object_info	Scene inspection	GET /objects/{name}	No
3	create_object	Object manipulation	POST /objects	Yes
4	modify_object	Object manipulation	PATCH /objects/{name}	No
5	delete_object	Object manipulation	DELETE /objects/{name}	No
6	set_material	Material control	PUT /objects/{name}/material	Yes
7	set_texture	Material control	PUT /objects/{name}/texture	No
8	execute_blender_code	Arbitrary Python - UNSANDBOXED	POST /exec	Yes (omit in prod)
9	get_viewport_screenshot	Viewport capture	GET /screenshot	No
10	get_polyhaven_category	Poly Haven assets	GET /polyhaven/categories	No
11	search_polyhaven_asset	Poly Haven assets	GET /polyhaven/search	No
12	download_polyhaven_asset	Poly Haven assets	POST /polyhaven/download	No
13	search_sketchfab_model	Sketchfab assets	GET /sketchfab/search	No
14	generate_hyper3d_model	Hyper3D Rodin / Hunyuan3D	POST /hyper3d/generate	No

Upstream also names `get_viewport_screenshot` as `get_blender_screenshot`, and ships `download_sketchfab_model` / `poll_rodin_job_status` / `import_generated_asset` as companions to rows 13-14.

Production advice: ship a 3-operation schema (`get_scene_info`, `create_object`, `set_material`) and leave `/exec` out entirely. Add it only to a separate private plugin for trusted operators - see [S11](#).

5. Three-Step Mac Setup

Prerequisites: **Blender 3.0+**, **Python 3.10+**, and the **uv** package manager.

Step 1 - Install uv

```
```bash brew install uv # or, without Homebrew: curl -LsSf https://astral.sh/uv/install.sh | sh uv --version ```
```

### Step 2 - Install the Blender addon

1. Download `addon.py` from the upstream repository.
2. In Blender: **Edit > Preferences > Add-ons > Install...**, select `addon.py`, then tick **Interface: Blender MCP** to enable it.
3. In the 3D Viewport press **N** to open the sidebar, choose the **BlenderMCP** tab, and click **Connect to MCP server**. The addon now listens on `localhost:9876`.

### Step 3 - Register the MCP server with your local client

Note the **absence of a -- separator** - this is the corrected form:

```
```bash claude mcp add blender uvx blender-mcp ```
```

Verify the socket is live before going further:

```
```bash nc -vz 127.0.0.1 9876 # expect: succeeded! ```
```

At this point hops 3-5 of the call chain work locally. Hops 1-2 require the HTTPS bridge and an OnDemand plugin registration, covered next.

## 6. The HTTPS Bridge

---

A minimal wrapper translating REST into the blender-mcp TCP JSON protocol. Run it on the same host as Blender, behind TLS.

```
```python # blender_bridge.py - HTTPS wrapper in front of the local blender-mcp TCP
socket server. import json, os, socket from flask import Flask, jsonify, request, abort
BLENDER_HOST, BLENDER_PORT = '127.0.0.1', 9876 BRIDGE_KEY =
os.environ['BRIDGE_KEY'] # shared secret, Creator Defined + Masked app =
Flask('blender_bridge') def send_to_blender(command, params): with
socket.create_connection((BLENDER_HOST, BLENDER_PORT), timeout=120) as sock:
sock.sendall(json.dumps({'type': command, 'params': params}).encode()) return
json.loads(sock.recv(1 << 20).decode()) @app.before_request def guard(): if
request.headers.get('X-Bridge-Key') != BRIDGE_KEY: abort(401) @app.get('/scene') def
get_scene_info(): return jsonify(send_to_blender('get_scene_info', {})) ```
```

Declare the shared secret as an apiKey security scheme in the schema with a matching action.fields[] entry so OnDemand injects the header on every call - "site": "header", "required": true, "editable": false. Set it as **Creator Defined** and **Masked** via creatorPluginConfig.fields.

7. OnDemand Registration

Registration is a single POST /plugin/v1 carrying your OpenAPI document. The header is exactly apikey (lowercase) - **not** Authorization: Bearer.

7.1 curl

```
``bash curl -sS -X POST 'https://api.on-demand.io/plugin/v1' \ -H "apikey:
$ON_DEMAND_API_KEY" \ -H 'Content-Type: application/json' \ -d
@plugin_payload.json ``
```

7.2 Python

```
``python import json, os, requests schema = open('blender_bridge_openapi.json').read()
# 4 operations, 8,969 bytes payload = { 'title': 'Blender MCP Bridge', 'description':
'Inspect and manipulate a live Blender scene over an HTTPS bridge.', 'action': {'type':
'openapi', 'schema': schema}, # NOTE: server wants a STRING 'creatorPluginConfig':
{'fields': [ {'name': 'X-Bridge-Key', 'site': 'header', 'required': True, 'editable': False,
'masked': True}, ]}, } r = requests.post('https://api.on-demand.io/plugin/v1',
headers={'apikey': os.environ['ON_DEMAND_API_KEY']}, json=payload, timeout=60)
print(r.status_code, r.text) # The 200 response has NO documented body schema -> do
NOT assume a pluginId here. # Read the plugin- ID from GET /plugin/v1/list instead. ``
```

Once a pluginId exists, attach it at session creation or on the query itself. Query-level pluginIds

replaces the session list - it is not merged, so include every needed ID.

8. Fact-Check Table

Every externally-sourced claim in the original draft was re-verified. Star count, licence and release facts are sourced from github.com/ahujasid/blender-mcp.

Claim	Verdict	Verified value
Claimed 16.9k stars	CORRECTED	Verified ~22.7k
MIT licence + actively maintained	CONFIRMED	PyPI v1.5.6 (18 Mar 2026)
claude mcp add blender -- uvx blender-mcp	CORRECTED	Corrected to claude mcp add blender uvx blender-mcp (no -- separator)
addon + sidebar panel + uv + localhost:9876	CONFIRMED	Two-process architecture verified
"ClaudeLog - Blender MCP" source	UNVERIFIED	Page could not be located
AI Architects guide (Tom Crawshaw, Jun 2026)	REAL	Corroborates the two-component architecture
arjun988_blender-skills pack	NOT_FOUND	Could not be retrieved from any source
Registration status	FAILED	POST /plugin/v1 -> HTTP 400 invalid_request, no pluginId issued

Because the arjun988_blender-skills pack could not be retrieved, **no skill-to-tool mapping table for that pack is reproduced in this guide**. A prior draft carried a 94-row reconstruction; it has been withheld rather than presented as sourced material.

10. Live Registration Record

REGISTRATION FAILED — HTTP 400 invalid_request · pluginId NOT ISSUED

Registration of the Blender MCP Bridge plugin **was actually attempted against the live OnDemand API on 2026-07-28**, using the real REST API key held in the ON_DEMAND_API_KEY environment variable. The attempt **FAILED with HTTP 400** and **no pluginId was ever issued**, so nothing in S7 should be read as a completed registration. The failure is a **server-side contract bug** - not a client error and not an authentication problem: the same key successfully created a real chat session (HTTP **201**, session 6a684c26a745cb4842705a44), proving the credential is valid and prod-scoped. The run left **no side effects** - a follow-up GET /plugin/v1/list confirms **0 plugins created** - and the OpenAPI document itself is sound (4 operations, 8,969 bytes, openapi-spec-validator **PASS**).

Field	Value
Status	FAILED — HTTP 400 invalid_request
Endpoint	POST /plugin/v1
Host used	https://api.on-demand.io
pluginId	NOT ISSUED
Fallback host	https://gateway-dev.on-demand.io → HTTP 401 (key is prod-scoped)
Verification session	6a684c26a745cb4842705a44 (HTTP 201 — proves the key works)
tool_invoked	false
Side effects	none — GET /plugin/v1/list confirms 0 plugins created
Schema validated	4 operations / 8,969 bytes / openapi-spec-validator PASS
Executed at (UTC)	2026-07-28T06:24:34Z

Verbatim record

```
apikey: <REDACTED> (env ON_DEMAND_API_KEY, 32 chars, never printed)
endpoint: POST /plugin/v1
host_used: https://api.on-demand.io
http_status: 400
error_body: {"message":"schema is required","errorCode":"invalid_request"}
pluginId: NOT ISSUED - registration FAILED
fallback_host: https://gateway-dev.on-demand.io → HTTP 401 (key is prod-scoped)
root_cause: sending action.schema as an object triggers a Go unmarshal error naming
             PluginAction.action.schema of type string - proving the field name is correct
             and a STRING is required. This is a server-side contract bug, not auth and not
             payload shape. ~25 real diagnostic POSTs isolated this.
verification_session_id: 6a684c26a745cb4842705a44 (chat session really created, HTTP 201 - proves the key works)
```

```
tool_invoked: false (no pluginId exists, so Step-2 tool verification was not run)
side_effects: none - GET /plugin/v1/list confirms 0 plugins created
schema_validated: 4 operations / 8,969 bytes / openapi-spec-validator PASS
executed_at_utc: 2026-07-28T06:24:34Z
```

What to do next

Open a support ticket with OnDemand quoting the exact contradictory pair, because the two errors cannot both be satisfiable: sending action.schema as a valid **8,969-character string** returns schema is required, while sending the very same document as an **object** returns json: cannot unmarshal object into Go struct field PluginAction.action.schema of type string. The second message proves the field name is right and that the server itself wants a string - so the first message is a server-side validation bug, not a payload mistake.

Until it is fixed, use the **console-UI fallback**: My Agents > Create Agents > paste (or Import from URL) the OpenAPI schema, add the X-Bridge-Key configuration field, save - then read the resulting plugin-... ID from GET /plugin/v1/list and use it in pluginIds.

11. Security Hardening

⚠ **execute_blender_code** executes arbitrary Python inside the Blender process, with **NO sandbox**. It is the single most dangerous surface in this integration.

- Full bpy access and full Python: os, subprocess, open(), socket. It can read or destroy any file the Blender user can touch and open outbound connections.
- Upstream has open issues covering exactly this: arbitrary exec() with no sandbox, and **weak file-path validation** in the screenshot / Hyper3D tools enabling **exfiltration**.
- **Prompt injection becomes remote code execution**. Any text the model ingests - a user message, a fetched web page, an asset description - can attempt to steer it into running attacker-chosen Python. The LLM is the only thing between untrusted text and exec().
- **Telemetry is ON by default**, capturing prompts, code and screenshots, with an open GDPR / opt-in issue. Disable it via preferences or environment variable.

Mitigations, in order of effectiveness

#	Mitigation	Rationale
1	Omit /exec from the registered schema entirely	If the tool is not in the OpenAPI document, the agent cannot call it. Correct default for anything user-facing.
2	Isolate it in a separate, unpublished plugin	Trusted internal operators only; return 403 code_execution_disabled by default.
3	Run Blender in a disposable, network-isolated container	No credentials mounted, no access to production storage.
4	Allow-list, don't deny-list	Permit a small set of known-safe operations - Python has too many escapes for a deny-list to hold.
5	Log every executed snippet	Record with the session / message ID for audit.
6	Never mix with untrusted web content	Do not combine execute_blender_code with agents that ingest untrusted web content in the same session.

Transport hardening

1. **Keep the Blender socket on loopback**. The addon listens on localhost:9876; never bind it to

0.0.0.0. Only the bridge - on the same host - connects to it.

2. **Validate Origin / Host on the bridge.** The MCP specification requires servers to validate the Origin header to mitigate DNS-rebinding attacks. Reject unexpected origins.
3. **Require a shared secret** (X-Bridge-Key) declared as an apiKey security scheme, Creator Defined and Masked.
4. **Allow-list egress** to OnDemand's callers where your infrastructure permits.

Not verifiable from the OnDemand live docs: OnDemand publishes no sandboxing, egress filtering, or code-execution policy for REST API Agent calls. Assume none exists; all containment is your responsibility, on your side of the bridge.

12. Troubleshooting

Symptom	Most likely cause	Fix
401 unauthenticated	Wrong header name	Header is exactly apikey (lowercase) - not Authorization: Bearer
404 not_found on a valid path	Wrong host for that service	Chat: api.on-demand.io; Flow Builder: api.on-demand.io/automation/api; keys/plugins: gateway-dev.on-demand.io
Model answers but never calls the tool	pluginIds absent at session and query level	Send pluginIds on the query, or set it at session creation
Tool worked, then stopped	Query-level pluginIds replaces the session list	Include every needed ID in the query array - they are not merged
Tool still not called, pluginIds present	fulfillmentOnly: true	Remove it, or set false
KeyError: 'sessionId' after create	Create returns data.id	Read data.id; only submitQuery returns data.sessionId
504 plugin_execution_timeout	Bridge slower than 2.5 min	Make the operation async: return a job ID and poll
502 from your bridge	Blender not running, addon not started, or wrong port	Start Blender, open the BlenderMCP sidebar (press N), connect; confirm port 9876
206 on /plugin/v1/list	Plugin subscription not active	Read message; subscribe / activate the plugin
429 rate_limit_exceeded	100 RAG calls/min exceeded	Back off; request a limit increase
Plugin rejected at publish	Schema >5 MB, >10 operations, missing descriptions, <2 conversation starters	Meet the publishing rules before retrying
Works locally, fails from OnDemand	Bridge not publicly reachable / bad TLS / rotated tunnel host	Use a stable HTTPS hostname with a valid certificate
400 'schema is required' on POST /plugin/v1	Server-side contract bug (see S10)	Use the console-UI fallback and read the ID from GET / plugin/v1/list

Final checklist

- Bridge reachable over public HTTPS with a valid certificate and a stable hostname
- Blender socket bound to 127.0.0.1:9876 only
- X-Bridge-Key declared, Creator Defined, Masked
- `execute_blender_code` omitted from the published schema, or isolated per S11
- Telemetry disabled
- Schema under 5 MB and 10 operations, every operation described
- Long-running operations made asynchronous (job ID + poll)
- pluginIds supplied on every query that needs the tool

13. Sources

Item	Reference
Upstream project (stars, licence, releases)	github.com/ahujasid/blender-mcp
OnDemand docs index	docs.on-demand.io/lms.txt
OnDemand error reference	docs.on-demand.io/reference/errors
OnDemand rate limiting	docs.on-demand.io/docs/rate-limiting
Live registration record	/tmp/shared/REGISTRATION_RESULT.txt (reproduced verbatim in S10)

Live OnDemand API documentation fetched 2026-07-28T03:21:49Z and verified at 03:30:06Z. This document is a **rebuild** of an earlier 21-page draft whose binary was not recoverable; the narrative is derived from the updated source markdown guide.